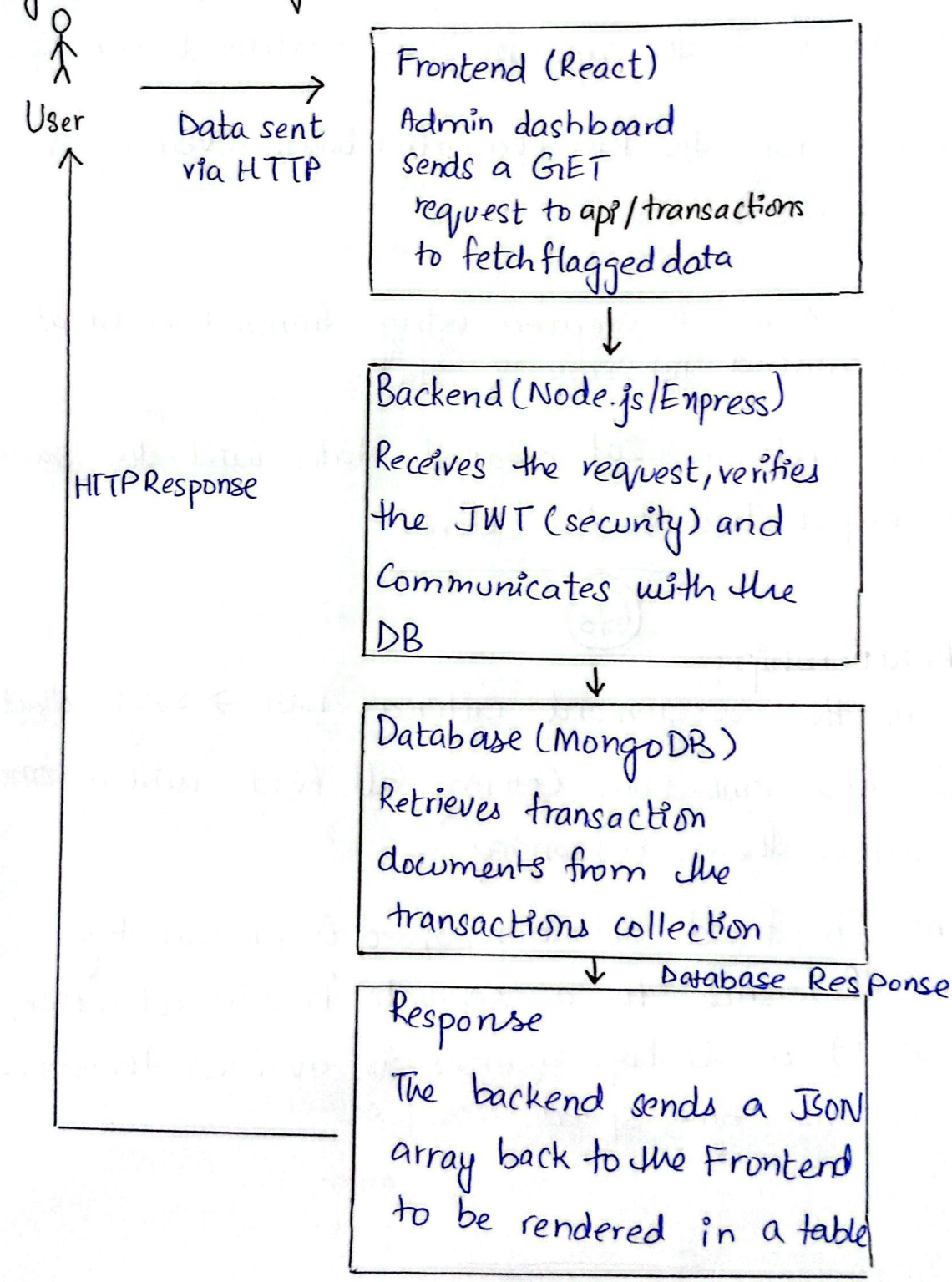
Name: Eman Faris
Section: 231-5572

Part D :-

Q1.

System Flow Diagram:



CamScanner

Q2.

One API Route

Endpoint: POST /api/transactions/evaluate

1. The user sends a JSON body containing { amount, location } to the server.

2. Middleware checks if the amount is a positive number.

3. The controller runs the Risk Evaluation logic (Velocity & Geographic checks.)

4. A new document is created using Transaction.create() with the calculated riskScore & status.

5. The server sends a 201 created status with the saved transaction object back to the client.

Q3.

Database Relationship:

According to the assignment criteria, basic → single collection. For this, I chose embedding (storing all fields within one document) rather than referencing.

Justification: In fraud detection, speed is critical. By avoiding a 'Reference' to a separate 'Results' collection, using .populate() or lookup queries is avoided therefore increasing response time speed.

CamScanner

Q4.

Core FinTech logic

My system uses a Weighted Risk Scoring model.

Example: city A = Islamabad , City B = Lahore

A transaction 12,000 occurs in city A. The following steps will be used:

1. Velocity ; Is 12,000 > 10,000? Yes (Add 50 points.)

2. Geographic; Is city A the user home city? (The user's home city is city B) so, NO. (Add 30 points.)

3. Total ; Current Score = 50+30 = 80 points

4. Decision; Since 80 > 70, the system automitically sets the status to "Blocked".

Q5.

Real Security Flaw

No Sql Injection, For instance if I used a raw query like Transaction.find({ transactionID: req.body.id }) , a malicious user could send {"id": { "$gt": "" }} in the requested body. This would bypass the specific ID check & return all transactions in the database.

Fix:

I implemented Input Sanitization using express-validator to ensure the ID is a plain string & I used Mongoose, which automatically schemas & cast filters to prevent most common injection attacks by ensuring the input matches the defined data type.

CamScanner